课程编号：A0801053010

分布式系统导论课程设计

雾桌（MistTable）多人在线虚拟桌游平台的技术分析和概要设计



组名	完成作业组	选题方向	B：多人实时协作的在线虚拟桌游平台
组长(100%)姓名	周宇斌	组长学号	20246085
副组长(98%)姓名	刘润七	副组长学号	20246162
副组长(98%)姓名	刘万轩	副组长学号	20246063
副组长(98%)姓名	周星宇	副组长学号	20245986
组员(80%)姓名和学号(不多于 4 人)	无		
评定成绩		评定人	
		评定日期	

东北大学软件学院

评分标准	得分
需求明确：清晰定义分布式系统的技术需求(25%)；	25
设计完整：涵盖大部分所学知识点(15%)；	15
设计正确：可以完成既定技术需求(30%)；	30
自主知识运用：是自己想的还是查阅资料获得的(20%)；	20
选题新颖：常规的教科书均可见的还是较为新颖的分布式系统(10%)；	10
大段文献抄袭且不给出引用的，对应考核点按零分处理	

目　录

1. 系统简介 4
   1.1 系统背景与业务定位 4
   1.2 为什么属于分布式系统 4
   1.3 核心业务功能与角色权限 4
   1.4 分布式技术需求与设计假设 5
   1.5 课件理论联系：分布式系统的目标、挑战与事务边界 6
2. 体系结构 6
   2.1 技术需求 6
   2.2 概要设计 7
      (1) 客户端—服务器分层协作架构 7
      (2) 房间级单权威处理路径与扩展方式 8
      (3) 核心架构选型对比：C/S 架构与全 P2P 架构的权衡 8
3. 进程 8
   3.1 技术需求 8
   3.2 概要设计 9
      (1) 客户端与服务端进程职责分工 9
      (2) 多房间分配与并行处理方式 9
      (3) 课件理论分析：并发模型、虚拟化与代码迁移适用性 10
4. 通信 10
   4.1 技术需求 10
   4.2 概要设计 10
      (1) 请求—应答与实时协作的通信分工 10
      (2) 角色标记移动与状态传播时序 11
      (3) 课件理论分析：瞬态通信 vs 持久通信、消息广播与恢复边界 12
5. 命名 12
   5.1 技术需求 12
   5.2 概要设计 12
      (1) 稳定实体标识设计 12

(2) 邀请码凭证与房间定位机制 13
(3) 课件理论分析：平面命名、结构化命名与 DHT/Chord 方案对比 13
6. 同步 14
6.1 技术需求 14
6.2 概要设计 14
(1) 基于房间权威定序与对象版本的冲突处理机制 14
(2) 掷骰与状态判定的有序性保障 15
(3) 课件理论分析：物理时钟偏差、逻辑时钟与并发控制算法比较 15
7. 一致性与复制 17
7.1 技术需求 17
7.2 概要设计 17
(1) 业务分级一致性模型构建 17
(2) 战争迷雾与成员权限的状态投影隔离 18
(3) 课件理论分析：主备复制与 Quorum 仲裁机制对比 19
8. 容错性与故障恢复 20
8.1 技术需求 20
8.2 概要设计 20
(1) 网络超时分析与基于操作唯一标识的幂等重试机制 20
(2) 客户端断线重连与增量/全量快照差分恢复流程 20
(3) 协作服务进程故障接管与恢复前提边界 22
(4) 课件理论分析：故障分类模型、可靠通信与 2PC/3PC 适用边界 22
9. 总结 23
9.1 系统设计与分布式知识点对应回顾 23
9.2 核心设计取舍与工程理性总结 24
9.3 设计局限性与未验证假设声明 24
参考文献 25
附录 (得分栏勿填) 26

1. 系统简介

1.1 系统背景与业务定位

在计算机支持的协同工作（CSCW）与线上互动娱乐领域，桌面角色扮演游戏（Tabletop Role-Playing Games, TTRPG，如跑团等）正在由线下实体聚会转向通过网络虚拟桌面（Virtual Tabletop, VTT）展开协同。参考优秀的轻量化虚拟桌游实践（如 Owlbear Rodeo [9]），本课程设计提出“雾桌（MistTable）”多人实时协作在线虚拟桌游平台。雾桌旨在通过纯浏览器终端，为主持人（Host）与玩家（Player）提供一个支持多人在线协同查看场景、移动角色标记（Token）、投掷骰子并由主持人控制战争迷雾（Fog of War）探索视野的在线跑团环境。

本概要设计报告的核心任务，并非实现一个具备支付、音视频或复杂商业规则系统的完整商业平台，而是围绕分布式系统的核心理论，系统阐述身处不同网络节点的多个用户如何共同操作同一个虚拟桌游房间，并在网络延迟、并发冲突以及局部节点异常等条件下，依然获得可解释、可恢复的共享状态。报告将重点结合东北大学软件学院《分布式系统导论》课程教学大纲与宋杰老师课件（Chapter 1–8）[1] 展开推演。

1.2 为什么属于分布式系统

根据分布式系统经典定义（Tanenbaum 著作 [2] 及课程课件 Chapter 1 [1]），分布式系统是由通过网络连接的多个自治计算机进程组成的集合，对用户表现为一个统一的连贯系统。雾桌平台之所以属于典型的分布式系统，体现在以下几个根本特征：

（1）独立物理进程协作：系统由分布在不同地理位置的浏览器客户端进程、接入服务进程、房间协作服务进程以及后端持久化存储节点组成。各个节点运行在各自独立的物理设备之上，完全通过网络消息传递实现协同，不存在全局物理共享内存。

（2）网络不可靠性与非零延迟：节点之间的交互依托公共互联网，必然面临网络传输延迟、网络抖动、报文无序到达或丢失等现象。系统无法假定任何跨网络操作能够瞬间完成或绝对可靠。

（3）缺乏统一的全局物理时钟：各个玩家设备与服务器节点的本地物理时钟存在不可避免的时间偏差，系统无法直接以客户端本地时间戳作为判定操作先后的权威依据。

（4）并发操作与共享状态维护：多名玩家可能在极短时间内对同一标记或场景要素发起移动或修改请求，系统需要在分布式环境下定义合理的操作处理顺序，防止状态产生歧义或覆盖。

（5）独立局部故障（Partial Failure）：网络中个别客户端意外掉线或某服务节点异常，属于典型的局部故障。系统必须在局部节点异常时保障其余节点的正常交互，并为异常节点恢复提供可靠依据。

1.3 核心业务功能与角色权限

系统定义两类核心用户角色：主持人（Host）与玩家（Player）。主持人负责创建房间、选择地图、调整战争迷雾与场景状态，并控制成员权限；玩家加入房间后，可查看经授权可见的场景、移动所绑定的自身角色标记并进行掷骰检定。系统核心业务功能与分布式技术问题的映射如表 1-1 所示。

表 1-1 系统核心业务功能与分布式技术问题映射表

功能编号	业务功能名称	用户行为描述	对应的分布式系统技术问题
FR-01	房间生命周期管理	主持人创建房间、配置属性；玩家通过凭据加入或退出	逻辑命名、服务定位与状态初始化
FR-02	预置地图场景呈现	主持人选定预置底图并向房间呈现；成员加载底图	静态资源访问、共享状态初态分发
FR-03	角色标记实时协作	获授权成员拖拽角色标记并同步位置坐标	并发请求处理、操作权威定序、增量状态同步
FR-04	战争迷雾权限隔离	主持人控制可见区域，遮蔽未探索内容	服务端受限投影、视野访问控制与信息隔离
FR-05	掷骰检定与结果同步	玩家请求掷骰，系统生成结果并在房间成员间同步	权威随机生成、请求幂等处理、有序事件广播
FR-06	增量状态实时分发	已确认的状态变化及时向房间在线成员推送	实时通信通道、面向群组的消息广播
FR-07	断线重连与状态恢复	客户端因网络抖动离线后重连，恢复房间状态	断线重新认证、增量补发与当前状态快照恢复

1.4 分布式技术需求与设计假设

系统明确界定功能范围：系统仅支持主持人选择系统预置的标准地图资源，不包含用户上传自定义地图文件、格式转码及外部存储流程。系统暂不纳入语音视频通话、支付系统、第三方插件市场、复杂规则引擎、全网状对等拓扑（P2P Mesh）及跨地域容灾设计，以确保概要设计聚焦于核心协同机制。

在系统规模与设计目标方面，确立了五项核心设计假设指标，如表 1-2 所示。

表 1-2 系统性能与规模设计假设指标表

指标分类	设计目标 / 假设数值	指标口径与测量边界说明	设计含义与约束
单房间承载规模	最多 8 名同时在线成员 (含主持人)	单房间实时通信并发连接数 ≤8	面向小型协作，避免无必要的超大规模群组通信复杂度
初始系统总体规模	约 100 个活跃房间并行处理	系统在设计推演中假定承载约 100 个自治桌游房间	为多房间分配与并行处理提供规模参考基准
实时状态更新延迟	P95 目标 ≤300 ms	同地域正常网络下，服务端权威提交至在线端收到更新	要求通信机制采用长连接双向通道，避免低效的定时轮询
常规管理 API 延迟	P95 目标 ≤500 ms	服务端接收管理请求至返回响应时间 (不含地图资源下载)	REST 接口面向低频控制操作，采用轻量无状态处理

指标分类	设计目标 / 假设数值	指标口径与测量边界说明	设计含义与约束
网络断线恢复时间	授权状态恢复目标 ≤ 5 秒	普通规模房间在连接恢复后完成授权状态对齐的目标时间	要求服务端提供增量事件缓冲或快照差分恢复途径

【关键设计限定声明】: 上述表 1-2 中列出的各项指标数值，均为课程设计推演中的“设计假设与设计目标”，并非指导教师规定的硬性指标，亦未经真实系统的实现编码、压力测试或故障演练验证。报告在后续各章节的方案设计与分析中，均严格以此作为理论推导边界，不将其描述为已验证的实测性能结果。

1.5 课件理论联系：分布式系统的目标、挑战与事务边界

对照宋杰老师课件 Chapter 1（Introduction）关于分布式系统核心理论的论述 [1]，系统设计体现了如下原理映射：

1. 分布式系统目标与透明性权衡：（1）资源共享：实现不同终端用户对虚拟地图、标记位置与操作历史等共享数据的高效访问；（2）透明性（Transparency）：系统关注访问透明性（客户端通过统一协议与服务端通信，屏蔽底层传输差异）与位置透明性（用户通过房间标识访问房间，无需感知其服务物理位置）。同时，遵从课件关于透明度权衡的观点：桌游业务存在战争迷雾机制，系统必须对未探索区域实施显式的信息过滤，不能进行无差别的全数据透明；（3）开放性与可扩展性：采用标准数据交换格式，按房间组织状态，支持多房间并行扩展。

2. 分布式环境的设计挑战：课件强调网络并非完全可靠、延迟并非为零。系统设计将网络丢包、延迟抖动与偶发断线视为系统必须面对的前提，不在逻辑层假定网络消息具有即时性与绝对可靠性。

3. 事务概念与持久化边界界定：课件 Chapter 1 详细分析了事务的 ACID 特性 [1, 10]。本设计将单次操作引发的状态变更、房间递增序号及操作记录纳入单个持久化存储事务中提交。在此必须明确界定：数据库或存储层事务提交成功，仅代表该操作在服务端持久化边界内完成了保存；它与网络广播消息是否已成功送达所有在线客户端，是两个不同层面的分布式问题，不能混为一谈。

2. 体系结构

2.1 技术需求

在虚拟桌游多端协同环境中，体系结构设计必须能够支持实时状态同步、权限校验与多房间并发处理。核心技术需求包括：

（1）共享状态权威判定需求：桌游协同涉及角色位置、场景切换及投骰记录，系统需要对同一房间内被接受的合法操作给出一致、明确的处理顺序，避免多端状态出现歧义。

（2）主持人权限与信息隔离需求：TRPG 跑团依赖战争迷雾机制遮蔽未探索区域。隐藏在迷雾后的要素信息需在架构层面进行受限管理，防范客户端直接读取完整底层数据造成的透视作弊。

（3）多房间并行自治与扩展需求：在面对约 100 个活跃房间的设计假设下，不同房间的操作数据完全独立，架构应天然支持房间间的并行隔离，允许通过水平扩展服务能力满足

规模需求。

2.2 概要设计

(1) 客户端—服务器分层协作架构

系统采用分层客户端—服务器（Layered Client-Server）架构与事件通知相结合的风格 [1, 7]。系统整体划分为客户端层、接入层、应用与房间协作服务层、持久化与基础设施层四个逻辑层次。系统总体逻辑架构如图 1 所示。

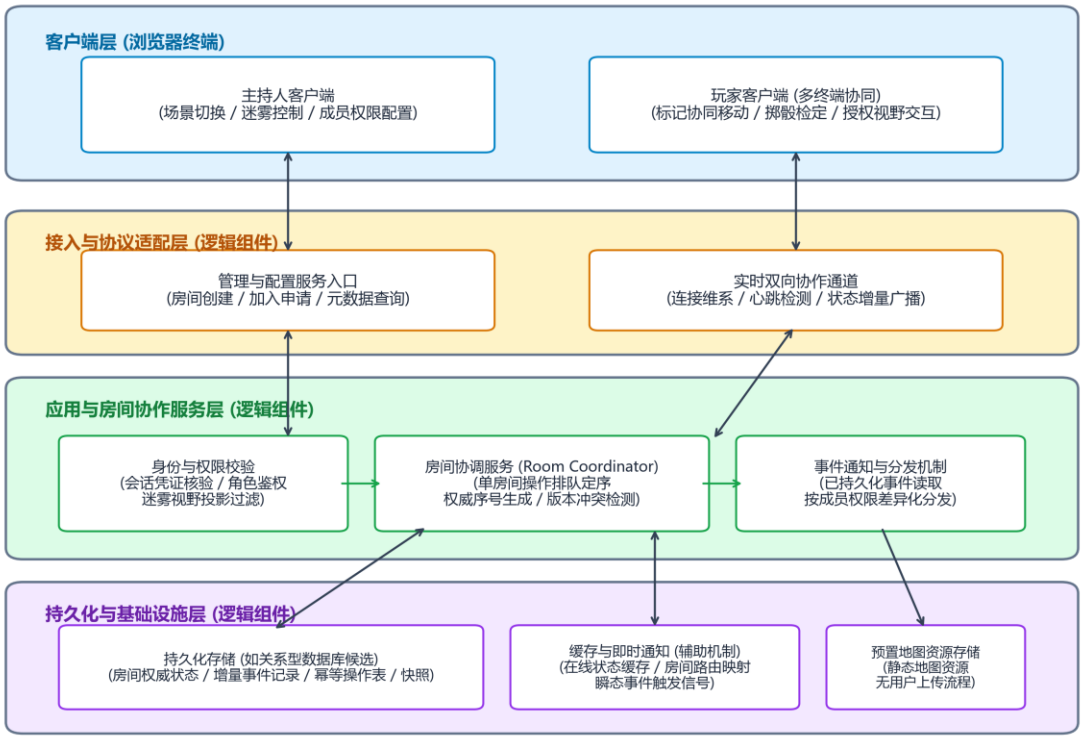


图 1　系统总体逻辑架构图

结合图 1，系统各逻辑组件的职责与调用边界如下：

1. 客户端层（Browser Clients）：包含主持人客户端与玩家客户端，运行于现代 Web 浏览器。负责渲染地图与图元、捕获用户交互、提供非权威的本地拖拽视觉预览，并通过网络向服务端提交操作请求。

2. 接入与协议适配层（Access Layer）：负责外部网络连接管理。包含处理常规房间管理与元数据查询的管理接口服务，以及承载长连接全双工通信的实时协作网关，承担身份凭证校验与连接保活工作。

3. 应用与房间协作服务层（Room Collaboration Service Tier）：系统的核心协作中枢。包含身份权限校验、房间路由及房间协作服务（Room Coordinator）。逻辑上，每个房间在任一时刻由单一权威路径负责排队处理操作、校验规则、检测冲突并生成递增事件序号；同时运行视野过滤逻辑，将房间内部状态转换为不同角色的可见视图。

4. 持久化与基础设施层（Storage & Infrastructure Tier）：持久化存储（如采用关系型数据库作为候选技术）负责持久保存房间状态、增量事件记录及操作幂等记录；辅助缓存组件用于维护在线成员列表与路由信息；预置底图资源由静态资源存储服务提供。

(2) 房间级单权威处理路径与扩展方式

为了兼顾单房间状态的一致性与全系统的处理能力，系统确立了‘不同房间并行处理，单房间权威定序’的设计思路：（1）房间间并行隔离：不同房间之间不存在数据依赖与共享状态，系统可按房间标识（roomId）将不同房间分配给不同的协作服务实例独立处理，支持通过横向扩容满足多房间负载；（2）单房间权威路径：对于特定房间内部的共享写操作，在逻辑上由一个权威处理路径顺序处理，避免了同一房间多个写入者之间的复杂分布式锁竞争，有助于保证操作顺序的清晰与可解释。

(3) 核心架构选型对比：C/S 架构与全 P2P 架构的权衡

在系统架构选型中，对等网络（Peer-to-Peer, P2P / WebRTC Mesh）常被用于实时协同。系统对分层 C/S 与 P2P 进行了对比分析，如表 2-1 所示。

表 2-1 客户端—服务器（C/S）与对等网络（P2P）架构方案对比表

评估维度	分层客户端—服务器架构 (C/S，当前方案)	对等网络架构 (P2P / Mesh，备选方案)
权威状态管理	服务端统一仲裁与落盘，规则校验与防作弊边界清晰	节点间对等同步，缺少中心权威，需依赖共识算法或 CRDT [13] 消解冲突
战争迷雾信息隔离	服务端依据权限计算视野差分投影，未探索数据在网络层被过滤	若全网分发底图依赖前端遮罩，存在客户端本地内存提取未探索要素的风险
网络连接拓扑	星型拓扑：客户端仅与服务端保持连接，单端连接开销较小	全网状网格：N 人房间需维护 N(N-1)/2 条连接，对弱网节点上行带宽压力大
断线状态恢复	服务端存储作为权威记录，掉线用户重连可从服务端恢复状态	房主或核心对等节点掉线可能造成状态丢失，选主与数据恢复协调复杂
架构取舍与代价	服务端承担网络与计算开销，是关键依赖，需设计容错与恢复机制	去中心化降低了中心服务器带宽负担，但协同逻辑与安全控制显著复杂

由表 2-1 可见，虽然 P2P 架构具备去中心化与节省服务端带宽的优点，但桌游平台具备显著的信息不对称性（战争迷雾）与单房间多用户强因果交互特征。P2P 架构在处理权限安全隔离、状态冲突收敛（如引入无冲突复制数据类型 CRDT [13]）与断线恢复方面需要付出极高的算法与协调代价。分层 C/S 架构以明确的服务端权威较好地契合了业务需求，代价是服务端成为关键依赖，需在后续章节重点展开进程、通信与容错设计。

3. 进程

3.1 技术需求

在进程组织与并发处理层面，系统面临如下技术需求与约束：

（1）连接承载与并发处理需求：接入服务需承载多客户端并发长连接，必须采用高效的并发 I/O 模型，避免因客户端等待或网络慢速传输耗尽系统线程资源。

（2）前端交互响应与计算解耦：浏览器端需处理图形渲染与网络事件响应，UI 主线程应避免被大开销的本地计算阻塞，维持交互流畅性。

（3）多房间进程调度与隔离需求：单个房间内的操作处理与密集计算不应干扰其他独立房间的正常协同。

3.2 概要设计

(1) 客户端与服务端进程职责分工

系统确立了‘客户端负责交互与渲染展现，服务端负责规则核验与权威状态维护’的职责划分 [1, 2]。职责分工如表 3-1 所示。

表 3-1 客户端进程与服务端协作进程职责分工表

处理阶段	浏览器客户端进程职责	服务端协作进程职责
用户交互与指令发起	捕获用户拖拽输入，生成非权威视觉预览，构造操作请求发送	接收操作请求报文，进行初步格式检验，不直接信任前端坐标
权限校验与规则判定	基于本地已知状态做必要的前置防呆校验 (非安全防线)	权威核验会话身份、角色归属、移动距离合法性及迷雾阻挡
并发控制与版本判定	提交请求附带本地基准版本号 (base_version)	基于当前对象状态进行版本匹配检查，拒绝过期操作
状态确认与序号赋予	无权擅自落地共享状态，维持临时未确认状态	确认合法操作并更新状态，为有效事件分配房间自增序号
视野投影与事件下发	接收经权限过滤后的更新，更新本地画布，清理无效对象	根据当前战争迷雾多边形剪裁可见性，按角色分发状态事件

(2) 多房间分配与并行处理方式

针对约 100 个活跃房间的设计假设，多房间分配与并行处理示意如图 2 所示。

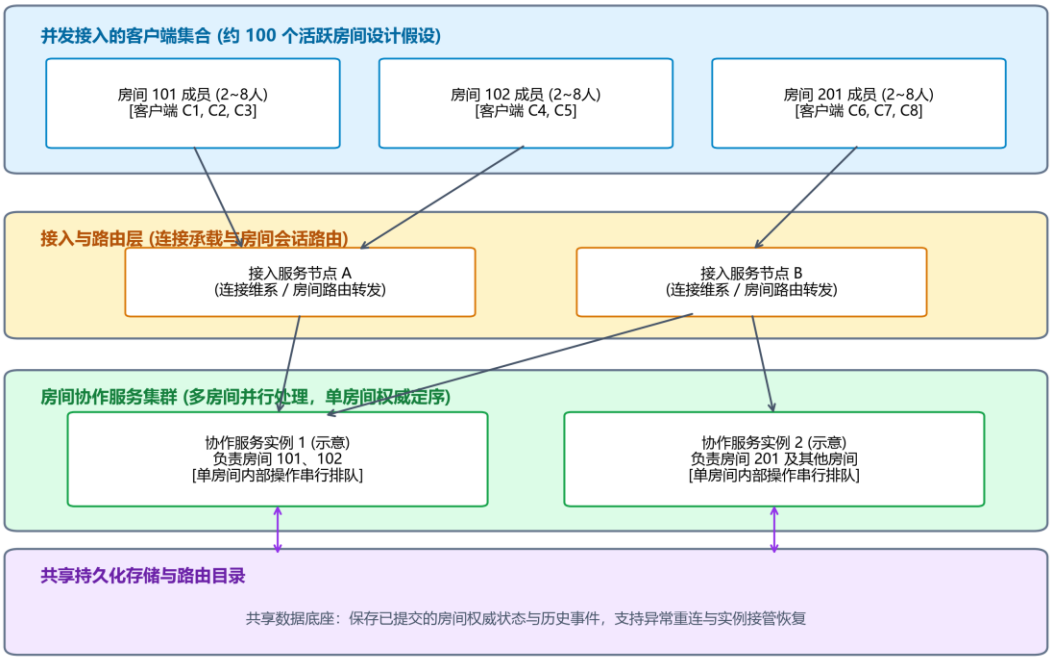


图 2 多房间分配与并行处理拓扑图

结合图 2，多房间处理调度逻辑如下：（1）无状态接入层转发：客户端连接接入服务节

点后，接入层仅维护连接链路，不保存业务状态，可根据房间标识将会话请求路由给负责该房间的协作服务节点；（2）房间自治处理：不同的房间可分配给不同的协作服务实例或不同工作单元并行处理；在单个房间内部，操作请求按到达顺序由权威路径排队执行，保证同一房间内的状态推进具备可解释的明确顺序。

(3) 课件理论分析：并发模型、虚拟化与代码迁移适用性

对照宋杰老师课件 Chapter 3（Processes）的核心概念 [1]，本设计对相关理论进行了如下分析：

1. 并发模型（Concurrency Models）：课件讨论了多线程与事件驱动模型的特性。在服务端，非阻塞事件循环适合处理大量并发连接的 I/O 多路复用，结合工作线程处理复杂的空间几何计算或存储操作，有助于提升吞吐量并防止主 I/O 阻塞；2. 虚拟化技术（Virtualization）：虚拟化与容器化可作为服务端组件的打包、隔离和部署工具，但它们属于底层运维支撑手段，不能自动替代应用层的状态管理、权限校验与容错逻辑；3. 动态代码迁移适用性分析（Code Migration）：课件详细介绍了代码迁移的概念与实现模式。在虚拟桌游平台中，客户端与服务端的职责边界固定且明确：客户端负责渲染交互，服务端负责安全校验与状态仲裁。业务上不存在在运行时将服务端代码动态推送到客户端执行，或者在不同节点间迁移包含执行上下文的进程的实际需求。强行引入代码迁移会增加系统的协议复杂度与安全攻击面。因此，系统判定动态代码迁移不适用于当前业务，不将其纳入核心设计。

4. 通信

4.1 技术需求

在虚拟桌游平台中，通信子系统负责节点间的信息交换，其面临如下技术需求与网络挑战：

（1）业务交互特征的分化需求：房间创建、配置读取等操作频率低、属于典型的单次交互；而拖拽标记、掷骰等交互频率高、要求低延迟的双向信息传递，通信体系需要支持差异化的协议分工。

（2）网络不可靠性应对需求：移动网络或公网环境下，消息可能面临传输延迟、意外丢包、报文乱序到达以及连接中断。通信协议必须考虑这些异常，不能假设消息一定会及时或成功到达。

（3）实时推送与历史恢复的分工需求：实时消息通道主要用于在线成员之间的快速状态扩散，而客户端掉线后的历史回溯则需要可靠的状态记录支撑，两者需合理衔接。

4.2 概要设计

(1) 请求—应答与实时协作的通信分工

系统采用‘请求—应答（Request-Reply）’与‘双向实时通信’相结合的通信架构 [1, 8]。不同业务场景下的通信方式对比如表 4-1 所示。

表 4-1 系统不同业务场景下的通信方式与协议特征对比表

业务交互场景	采用通信方式	通信交互模式	消息时效与持久性	设计理由与权衡
房间创建 / 加入验证	HTTPS / REST	请求—应答 (Request-Reply)	单次交互，服务端维护状态	语义明确，便于与常规认证和接入控制体系集成
预置底图元数据查询	HTTPS / REST	请求—应答 (Request-Reply)	静态资源支持本地与缓存机制	复用成熟的 Web 资源缓存机制，减轻实时通道负担
标记移动 / 掷骰指令	双向实时通道 (如 WebSocket)	双向全双工异步消息	即时发送，由服务端落盘确认	减少握手开销，提供较低的网络交互延迟
增量状态广播推送	双向实时通道 (如 WebSocket)	服务端主动单播 / 组播	瞬态推送，历史依赖存储	及时向在线授权成员同步最新变化，避免频繁轮询
断线重新同步与追齐	双向通道 / REST 辅助	请求—应答式补发	从持久化记录读取历史更新	基于序号获取落后事件或拉取当前授权快照

(2) 角色标记移动与状态传播时序

系统在处理角色标记移动时的权威提交与状态传播时序如图 3 所示。

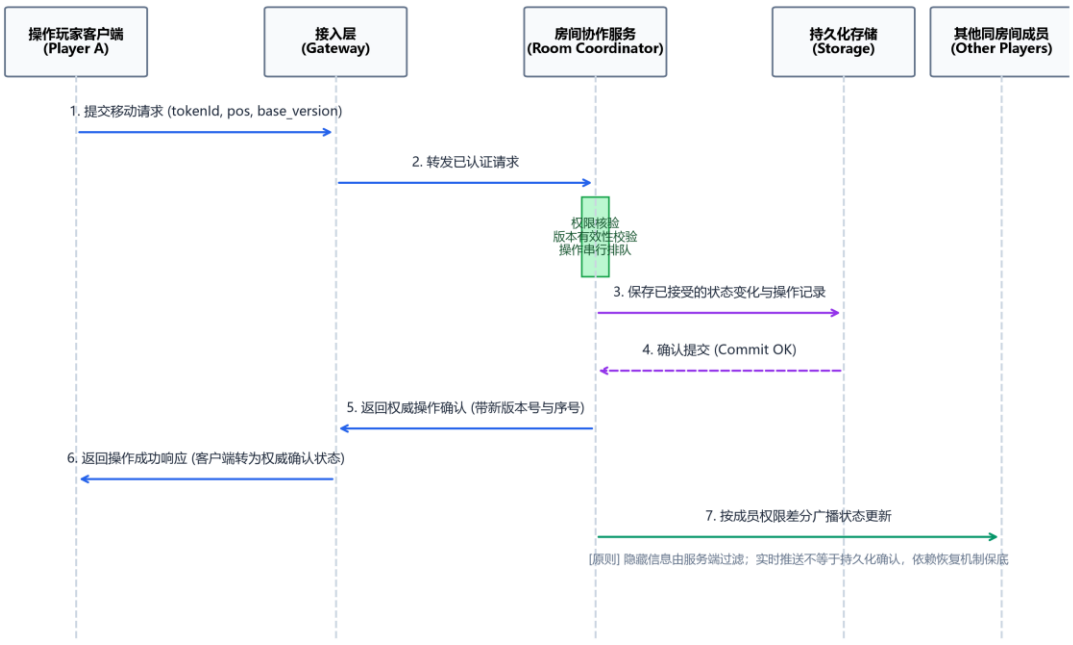


图 3 移动标记的权威提交与状态传播时序图

结合图 3，移动标记的核心协作时序流转如下：（1）操作发起与意图提交：玩家客户端

发起移动操作，提交所操作的角色标记标识、目标位置以及发起该操作时所依据的对象基准版本；（2）接入转发与会话路由：接入层网关在核验传输层连接与用户基础凭证后，将操作请求路由分发给负责该房间的协作服务节点；（3）权威核验与事务提交：房间协作服务作为单权威路径，校验玩家对该标记的控制权限以及基准版本的有效性。核验通过后，将新状态变更写入持久化存储事务，并生成该房间单调递增的事件序号；（4）操作结果确认：持久化写入确认后，服务端向发起方返回提交确认应答，驱动客户端将本地推测预览固化为权威状态；（5）权限投影与差异化传播：服务端结合当前战争迷雾区域，计算房间内其他成员对该标记移动事件的可见性。对于具备视野的成员进行增量事件推送；对于处于未探索迷雾中的成员，则在服务端完成消息过滤，不予下发。

(3) 课件理论分析：瞬态通信 vs 持久通信、消息广播与恢复边界

对照宋杰老师课件 Chapter 4（Communication）关于通信分类的论述 [1]，系统对消息机制进行了明确界定：

1. 瞬态通信（Transient Communication）的特性与局限：课件指出，瞬态通信仅在通信各方均正常运行时才能完成传递；若接收方正在经历断线或网络中断，消息往往被网络介质丢弃。WebSocket 属于典型的瞬态通信形式。系统明确界定：WebSocket 推送成功仅代表数据已写入发送缓冲区，绝不等于对端客户端已经确认接收，更不等于该数据已经持久化保存。

2. 持久通信（Persistent Communication）在恢复中的必要性：课件指出，持久通信在接收方未就绪时由存储中介保留消息。为了支持断线恢复，系统必须将已确认的关键状态变化写入持久化存储中保存。在线实时通道负责快速通知，而持久化存储记录则作为客户端断线后状态补齐的根本来源，二者互相配合形成完整的通信保障。

3. 广播机制的边界与可靠性假定：系统并不假定底层网络具备原子广播（Atomic Broadcast）或严格的可靠组通信机制 [1, 15]。广播消息在公网传输中可能遭遇偶发丢包。系统通过在事件中附带递增序号，使客户端能够感知到自身接收序列的不连续，并在重连或心跳检测中主动请求补齐，而非在传输层依赖开销巨大的多阶段组播确认协议。

5. 命名

5.1 技术需求

在分布式协同环境中，实体识别与资源寻址是系统运转的基础，命名需求体现在以下层面：

（1）全局实体唯一标识需求：用户、房间、角色标记及底图资源需要稳定、跨节点唯一的标识符，便于在网络交互与存储中精确引用。

（2）便于用户分享的房间凭据需求：玩家加入房间需要输入人类友好、便于在即时通信工具中传播的简短邀请凭证，且该凭证需与底层持久房间标识解耦，并具备可撤销性。

（3）网络物理位置透明性需求：服务端节点可能随维护或扩容发生变化，客户端请求不能与具体的物理 IP 或端口强绑定，需要提供灵活的服务定位与路由机制。

5.2 概要设计

(1) 稳定实体标识设计

系统采用全局唯一标识符（如 UUIDv4）作为业务实体的稳定内部名称 [1, 2]。UUID 具备极大的标识空间，支持在分布式节点上去中心化生成，避免了对中心自增发号器单点的强

依赖。核心实体标识包括用户标识（`userId`）、房间内部标识（`roomId`）、标记标识（`tokenId`）与地图资源标识（`mapId`）。需要特别明确的是：UUIDv4 仅用于全局实体的内部引用与寻址，其随机性绝不代表安全授权机制；知晓或获取某个实体标识绝不等于获得操作权限，随机标识符不能替代服务端严格的会话身份认证与角色权限核验。

此外，系统引入操作意图标识（`operationId`），由客户端在构造写操作时生成并附带，用于服务端在网络超时重试时识别重复请求并辅助实现幂等性。

(2) 邀请码凭证与房间定位机制

为了兼顾分享便利性与安全性，系统将‘加入凭证’与‘房间持久标识’清晰解耦：（1）房间邀请码（Join Code）：生成易读短码（如 6~8 位字母与数字组合）。邀请码仅作为申请加入房间的临时校验凭证，不作为数据库持久主键；服务端对邀请码进行安全哈希存储，并支持主持人随时重置邀请码使旧码作废；（2）房间服务定位与路由：系统在路由缓存中维护房间标识至负责服务实例的映射关系。客户端发起实时协同连接时，接入层依据该路由映射将请求定向至负责该房间的协作服务节点。系统核心命名规则如表 5-1 所示。

表 5-1 系统核心实体命名与服务寻址规则表

命名对象分类	采用标识形式	有效作用域	寻址与解析方式	设计目的与说明
业务持久实体 (Room/User/Token)	全局唯一标识 (如 UUIDv4)	全生命周期有效	数据库主键索引直查	去中心化生成，具备位置无关性；仅为标识，不能替代服务端权限核验
房间社交加入凭证	6~8 位字母数字短码	房间配置期有效，支持重置	通过哈希校验解析至 roomId	便于人类记忆与社交分享，哈希存储防泄露
操作意图标识 (Operation)	客户端生成唯一 ID	单次操作请求周期	服务端操作缓存表检索	用于识别超时重试请求，辅助实现幂等性
服务端网络物理地址	动态 IP:Port 组合	节点进程运行期有效	服务发现 / 路由目录查询	网络物理位置对客户端透明，便于节点动态调度
预置静态地图资源	结构化 URI 层次命名	静态资源长期有效	静态资源服务 / CDN 路由	便于分类管理与边缘缓存

(3) 课件理论分析：平面命名、结构化命名与 DHT/Chord 方案对比

对照宋杰老师课件 Chapter 5（Naming）的理论框架 [1]，系统对不同命名方案进行了分析：

1. 平面命名（Flat Naming）的适用性：课件指出平面标识符不包含位置信息。系统采用的 UUID 属于平面命名，其优势在于当房间在不同物理服务节点之间调度迁移时，其业务标识符无需发生变化，实现了良好的位置独立性。

2. 结构化命名（Structured Naming）在资源管理中的应用：课件介绍了树状层次化命名体系。系统在管理 API 与静态地图资源路径上采纳结构化 URI 命名（如 `/api/v1/rooms/{roomId}/tokens`），符合直观的逻辑从属关系，便于路由分发与分层管理。

3. 分布式哈希表（DHT / Chord）适用性权衡分析：课件 Chapter 5 详细讲授了 Chord 算

法及其基于哈希环与指位表（Finger Table）进行去中心化寻址的机制 [1, 14]。

方案分析：DHT/Chord 适合于节点规模巨大、频繁动态加入与退出的完全对等系统。然而在雾桌平台的应用场景下，系统初始设计假设承载约 100 个活跃房间，服务节点均属于中心受控的云端服务实例。若在百级规模下强行引入 Chord 覆盖网络，为了定位房间往往需要跨节点进行多跳（Hop）网络查询，不仅显著增加了交互延迟，而且需要额外维护复杂的节点加入、退出与指位表周期探测逻辑。相较之下，采用集中式的内存路由目录能够以极低的时延（一次查询）完成寻址，运维与实现成本也显著更低。因此，系统在技术比较后明确判定当前阶段不采用 DHT/Chord，体现了以需求为导向的合理取舍。

6. 同步

6.1 技术需求

在虚拟桌游的实时协同交互中，多人针对同一共享画板的并发操作不可避免，同步设计面临如下核心需求：

（1）空间标记并发修改的防静默覆盖需求：两名玩家可能在近似相同的微小时间窗口内试图移动同一个角色标记。系统不能允许后到达的网络请求无条件覆盖先达请求，必须提供可解释的冲突检测与状态仲裁机制。

（2）物理时钟不可靠下的排序需求：不同客户端设备的本地物理时钟存在几十毫秒至数秒不等的偏差，网络传输延迟亦不均衡。系统不能依赖客户端本地物理时间戳作为判断操作先后的权威依据。

（3）事件序列与因果关系的明确性需求：掷骰结果、角色位移与回合流转之间具有明确的因果关系，房间各成员端所呈现的事件演进历史应当具备一致的因果逻辑。

6.2 概要设计

(1) 基于房间权威定序与对象版本的冲突处理机制

为了在保证逻辑清晰的前提下实现并发控制，系统确立了‘单房间排队定序’与‘标记对象乐观版本检查（Version Check）’相结合的机制 [1, 2]。图 4 详细展示了两名玩家近似同时请求移动同一标记时的冲突检测与处理流程图。

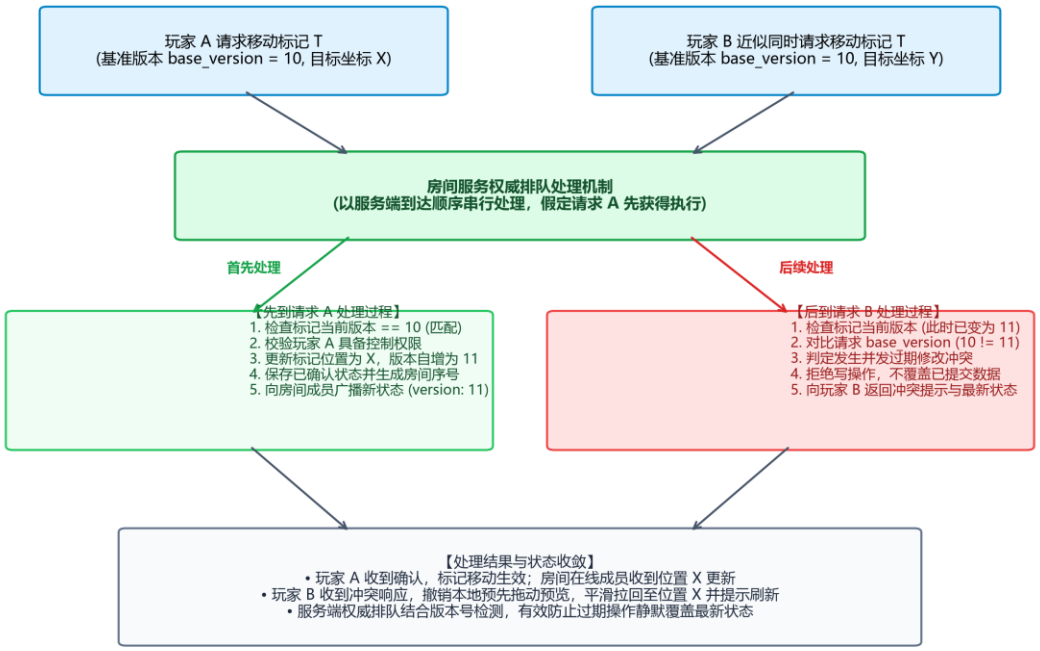


图 4　同一标记并发移动冲突与版本处理流程图

结合图 4，对并发冲突的处理逻辑阐述如下：（1）排队与意图核验：当多名玩家几乎同时请求移动同一标记时，客户端提交的操作请求携带发起操作时所依据的基准版本号（`base_version`）。请求到达服务端后，由负责该房间的协作服务按接收顺序依次排队处理；（2）先到请求成功提交：首个被处理的请求，其基准版本号与标记对象的当前权威版本相匹配。服务端接受该操作，更新坐标并递增对象版本号，生成房间内单调递增的事件序号，向全房间广播更新；（3）并发冲突检测与拦截：后续到达的操作由于其携带的基准版本已落后于当前已递增的权威版本，被服务端判定为基于过期视图发起的陈旧意图，服务端直接拦截并拒绝该次写入修改；（4）客户端状态平滑恢复：服务端向冲突方返回拒绝提示及该标记当前的最新权威状态。客户端收到响应后撤销本地预先拖拽的半透明视觉预览，平滑将标记重置回权威位置并更新本地版本显示。该机制有效防止了旧操作对新状态的静默覆盖，维护了房间内操作流转的全局全序。需要指出的是，单房间权威分配递增事件序号与基准版本校验，保障了已提交事件在所有在线成员端的统一观察全序；但在缺乏额外因果依赖追踪（如向量时钟或客户端未确认依赖图）的情况下，单房间全序不应被直接等同于完备的分布式因果一致性保证。

(2) 掷骰与状态判定的有序性保障

对于掷骰检定与回合流转，系统由服务端负责逻辑仲裁：客户端发送掷骰请求，服务端生成结果并在持久化存储中完成确认，赋予递增序号后向全房间广播。所有在线端依据接收到的事件流顺序记录投骰日志，确保房间成员观察到的投骰历史具备统一的逻辑全序先后关系。

(3) 课件理论分析：物理时钟偏差、逻辑时钟与并发控制算法比较

对照宋杰老师课件 Chapter 6（Synchronization）关于分布式时钟与互斥算法的理论[1]，系统对相关方案展开了对比分析，如表 6-1 所示：

表 6-1 分布式时钟与并发控制算法适用性对比分析表

同步 / 互斥技术机制	核心运作原理	在虚拟桌游系统中的适用性分析	采纳结论与设计权衡
物理时钟同步 (NTP)	通过网络时间协议周期校准节点物理时钟	由于网络传输时延抖动，节点间存在数十毫秒偏差	仅作为日志记录与监控参考，不作为并发操作定序的权威依据
Lamport 逻辑时钟 [3]	基于事件因果关系定义偏序关系 a -> b	课件经典理论模型，用于解释分布式因果依赖与偏序概念	与本系统定序机制明确区分：系统采用服务端分配单调事件序号实现房间内全序，理论模型用于阐释因果依赖
向量时钟 (Vector Clock) [6]	各节点维护向量数组，精确捕获并发因果	课件高级理论。支持无中心对等节点间的并发分支捕获	在单房间集中仲裁模型下元数据开销偏大，非核心实现所必需
单房间权威协调者定序	单房间由单一权威路径顺序排队处理	在服务端入口消解并发冲突，逻辑清晰且开销确定	【核心方案】：以清晰的工程思路实现房间内的操作全序定序
Ricart-Agrawala 分布式互斥 [5]	基于组播与应答的去中心化临界区访问	进入临界区需发送 N-1 条请求消息并接收 N-1 条应答消息，共 2(N-1) 条消息；任一节点失联均易造成阻塞	仅作课件学术对比，在存在中心服务器的体系下消息开销过大，不采用
令牌环互斥 (Token Ring)	逻辑环上传递令牌，持牌节点方可操作	令牌丢失恢复协议复杂，且难以适应浏览器客户端随时退出的情况	仅作课件学术对比，不适合动态 Web 客户端环境，不采用

深入理论剖析与概念澄清：课件详细证明了 Lamport 逻辑时钟 [3]、Ricart-Agrawala 分布式互斥算法 [5] 与向量时钟 [6] 在解决缺乏物理时钟因果定序方面的经典地位。在此需要明确区分分布式理论模型与本系统的工程实现机制：

第一，明确区分 Lamport 逻辑时钟与服务端递增事件序号：Lamport 逻辑时钟是分布式理论中用于在缺乏全局物理时钟条件下解释事件因果顺序（Happens-Before）与推导偏序关系的经典理论模型；而本设计在单房间内实际采用的排序方式，是由单房间权威协作服务在串行仲裁路径上统一分配的权威自增事件序号。前者在理论层面解释了因果依赖，后者则是工程上依靠集中权威仲裁为房间内所有操作提供明确全序的实现手段，二者机制不同，不可混同。

第二，互斥算法的架构适用性权衡：Ricart-Agrawala 算法适用于无中心的对等网络，但每次进入临界区需全网组播 N-1 条请求消息并等待接收 N-1 条应答消息，进入临界区前的消息总数为 2(N-1) 条，且对节点故障极度敏感；而在本系统选定的分层 C/S 架构下，服务端房间协调服务充当了天然的逻辑仲裁点，通过请求排队结合乐观版本校验即可低开销化解并发写冲突，无需在客户端之间运行开销巨大的分布式互斥协议。应当指出，单房间串行化处理牺牲了一定程度的单房间多操作并行能力，但对于通常不超过 8 人的跑团房间而

言，这种权衡能够极大简化逻辑并提升可靠性；全系统的扩展性则通过不同房间的并行处理来获得，二者不能混淆。

7. 一致性与复制

7.1 技术需求

在多人协同交互平台中，不同业务数据对一致性强度的要求存在客观差异。系统需要根据业务特征实施分级的一致性设计：

（1）核心对局状态的确定性需求：标记位置、血量数值、场景与投骰历史属于核心游戏事实，全房间成员观察到的已提交状态演进历史必须保持一致的因果顺序。

（2）客户端交互的及时响应需求：用户在操作界面拖拽时，本地界面应能即时响应预览，不能因为等待网络往返而出现画面迟滞，需要支持客户端中心一致性体验。

（3）非关键瞬态信息的最终收敛需求：鼠标指针轨迹（Cursor Presence）属于高频丢弃性数据，系统只需保证其在传输完成后达到最终收敛，容忍传输过程中的短暂延迟。

7.2 概要设计

(1) 业务分级一致性模型构建

对照课件 Chapter 7（Consistency and Replication）关于以数据为中心（Data-Centric）和以客户为中心（Client-Centric）的一致性模型分类 [1, 2]，系统构建了分级一致性策略，如表 7-1 所示。

表 7-1 系统各业务数据类型的一致性等级与保证策略表

数据业务分类	涉及典型实体	采纳一致性模型 / 交互机制	客户端感知体验	概要工程策略与保障前提
权威对局核心状态	标记位置、属性、迷雾区域、投骰记录	顺序一致性 (Sequential Consistency)	房间在线成员观察到全局一致的操作提交顺序	由协调服务排队定序，分配单调递增序号，客户端按序回放
本地操作交互体验	本端正在拖拽的标记、本端发起的移动	乐观交互预览 (Optimistic UI Preview)	拖拽即时视觉呈现，消除等待网络往返导致的交互迟滞	本地维护非权威推测状态，获服务端确认后固化；若冲突则拉回权威状态
客户端后续读取保证	本端后续查看房间、重连拉取快照	以客户为中心一致性 (Read-Your-Writes, Monotonic Reads)	已确认的操作在后续读取中立即可见，版本单调不减	客户端携带已确认事件序号水位，服务端快照拉取与增量同步保障不返回更旧状态
瞬态协同感知数据	成员鼠标光标位置、输入指示提示	最终一致性 (Eventual Consistency)	高频变动，短暂网络延迟下光标略有不同步，最终收敛	实时通道定期采样广播，最新位置覆盖旧数据，不入持久化事务

理论概念对应与严格区分：

（1）顺序一致性（Sequential Consistency）：分布式系统课件 Chapter 7 与经典教材 [1, 2] 将其定义为所有进程的操作执行结果与某种串行执行结果相同，且每个进程自身的操作在该序列中保持其程序顺序（最早由 Leslie Lamport 于 1979 年提出）。系统通过单房间协调服务为提交事件分配单调递增序号，客户端按序号回放，从而在工程上提供满足顺序一致性的体验。

（2）乐观交互预览（Optimistic UI Preview）与读自身写（Read-Your-Writes）的严格区分：在分布式交互设计中，必须严格区分客户端本地预览与分布式一致性保证的理论界限：① 乐观交互预览属于前端交互响应优化机制：客户端在用户拖拽标记时，不等待网络往返与服务端确认，立即在本地画布渲染半透明推测状态。该操作尚未经过服务端权威仲裁，一旦因并发版本冲突或权限不符被服务端拒绝，客户端必须撤销预览并回滚至服务端权威状态。因此，客户端提前显示一个尚未被服务端确认的操作，并不能单独证明系统满足读自身写；② 读自身写（Read-Your-Writes）则是经典的以客户为中心的一致性模型（Client-Centric Consistency）[1, 12]，其严格定义为‘一个进程完成写操作后，该进程随后的任何读取操作都将返回该写操作（或更新的写操作）更新后的值’。在雾桌平台的概要设计中，该保证依赖于服务端确认与读取水位的协同：首先，写操作必须经由单房间协调服务权威仲裁并持久化确认，向客户端返回该操作对应的单调递增序号（seq）；其次，客户端记录该序号作为本地已提交水位，在后续发起任何状态拉取、快照重置或断线恢复请求时携带该水位，服务端与协作网关确保返回的房间视图版本不低于该水位，从而保障玩家在操作确认后绝不会读到旧于自己修改的陈旧视图。必须客观指出，上述读自身写与单调读保证在当前阶段属于系统概要架构所确立的一致性设计要求与目标，其在极端弱网波动与复杂故障切换场景下的完备性，仍有待后续工程原型与全面集成测试进一步验证。

(2) 战争迷雾与成员权限的状态投影隔离

在虚拟桌游平台中，一致性并不等于‘所有客户端存储完全相同的数据’。战争迷雾机制决定了不同角色对场景的可见范围存在客观不对称性。图 5 展示了基于角色与迷雾规则的视图状态投影原理。

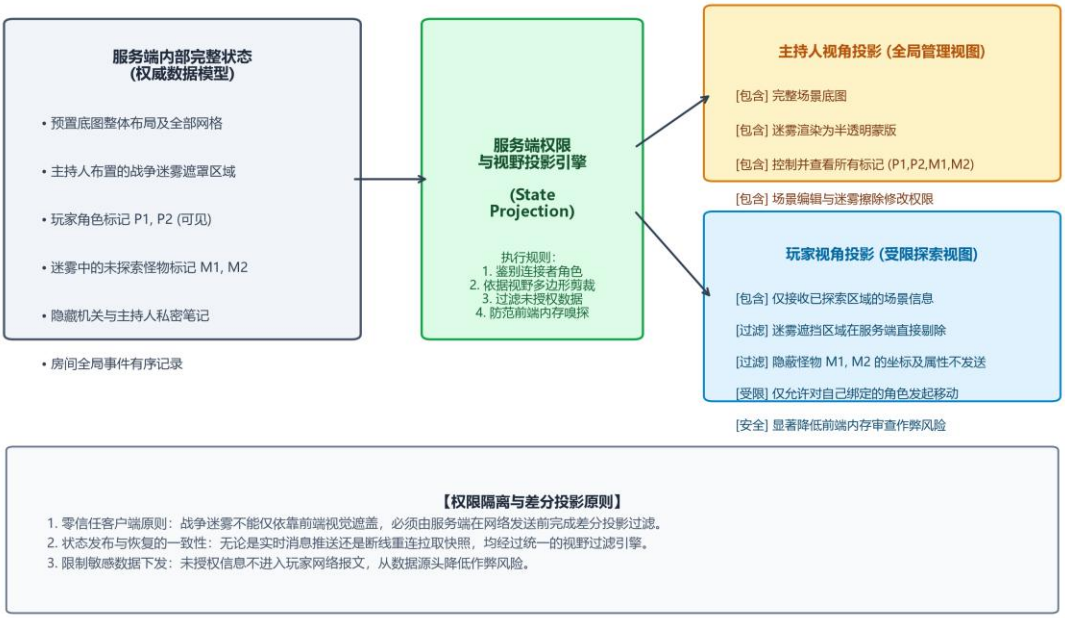


图 5　战争迷雾与角色权限状态投影图

结合图 5，权限隔离与状态投影的设计原则如下：（1）零信任客户端原则（Zero-Trust Client）：系统假设运行在浏览器端的代码处于不受信任的网络环境，战争迷雾不能仅仅依靠前端界面的视觉遮盖；（2）服务端状态投影：服务端维护房间的完整权威数据。在向具体客户端下发数据之前，服务端根据连接者角色与当前迷雾区域进行过滤裁剪：主持人获得全局完整场景；玩家端仅接收已探索区域内的场景和标记，未探索的怪物坐标与属性在网络报文构建阶段即被过滤，从数据源头降低客户端作弊风险；（3）实时广播与状态恢复的一致性：无论是实时增量推送还是断线重连拉取快照，均经过相同的权限投影逻辑，保证信息隔离规则在各种通信路径下的统一。

(3) 课件理论分析：主备复制与 Quorum 仲裁机制对比

对照宋杰老师课件 Chapter 7 关于多副本复制协议的理论 [1]，系统对主备复制与 Quorum 机制进行了对比论证：

1. 主备复制模式（Primary-Backup）：课件中主备复制由主节点接收写操作，并负责将日志同步至备用节点。该模式与系统‘单房间单权威协调’的逻辑拓扑相契合，读写路径清晰，成熟度高；2. 基于法定人数仲裁的 Quorum 投票机制对比：课件详细推导了 Gifford 的 Quorum 模型 [1, 4]。需要特别指出的是，文献与教学中常见的法定人数约束条件 $R + W > N$ 且 $W > N/2$，属于所有副本赋予相同权重（等权投票，即每个副本计 1 票）时的特化简化条件；而在一般的加权投票模型（Weighted Voting）中，各个节点可依据性能或网络可靠性分配不同票数权重 v_i，法定人数要求读票数总和与写票数总和严格大于总票数 V（即 $R + W > V$），且写票数严格大于总票数的一半（$W > V/2$）。Quorum 模型适合在无中心分布式存储中容忍部分副本故障。但在桌游协作场景下，若在应用层实现 Quorum 投票，会导致每次写操作都需要多节点网络往返协调，不仅显著增加端到端延迟，而且增加了系统复杂度；3. 复制边界与数据丢失窗口说明：系统倾向于采用成熟存储设施的主备复制方案。然而必须客观声明：当主备节点采用异步复制时，在主节点遭遇意外掉电崩溃的极端故障瞬间，未完全同步至备节点的微小增量存在丢失窗口。系统在当前概要设计阶段不作

无条件的‘零数据丢失’承诺，一切以实际部署与容灾配置为准。

8. 容错性与故障恢复

8.1 技术需求

在分布式网络环境下，局部故障是不可避免的常态。容错子系统必须具备应对以下典型异常的设计考量与恢复能力：

（1）弱网抖动下的状态对齐与恢复需求：玩家在无线网络或移动网络下面临短暂断连、丢包与延迟波动。系统需要设计状态判定与补发机制，在网络连通后使客户端尽快恢复到权威视图。需要强调的是，‘网络恢复后普通规模房间在 5 秒内恢复授权状态’属于未经压力测试的设计假设目标，用于指导差分恢复策略设计，而非经过实测验证的性能承诺。

（2）网络超时不确定性下的防重复操作与幂等需求：客户端提交掷骰或标记更新请求后，由于下行网络拥塞导致响应丢失，客户端在超时后可能发起重试。系统必须避免因网络重试导致操作被重复执行（例如重复扣减角色生命值或重复记录掷骰结果），该机制严格依赖持久化操作记录与本地事务边界。

（3）服务实例异常退出时的状态接管与防脑裂需求：负责特定房间的协作服务实例发生异常或失联时，备用接管实例需要从权威持久化存储拉取最新状态并恢复服务。在接管过程中，必须具备防止原实例与新实例同时写入的隔离机制（Fencing），避免产生双主（Split-Brain）数据混乱。

8.2 概要设计

(1) 网络超时分析与基于操作唯一标识的幂等重试机制

在分布式系统理论中，‘请求超时（Timeout）并不等同于操作执行失败’[1, 2]。当客户端发起掷骰请求且未在预期时间内收到确认响应时，实际物理网络中可能存在三种不同情况：① 请求报文在上传途中丢失，服务端未收到；② 服务端已成功接收并执行了本地事务提交，但在返回确认响应包时下行网络发生丢包；③ 服务端正处于高负载排队中，请求正在处理但尚未完成响应。

针对上述网络不确定性，系统设计了基于唯一标识符（`operationId`）的幂等重试机制：客户端在构造写操作请求时附加全局唯一标识符；服务端在持久化存储中，将业务状态变更与该操作标识在同一个本地事务边界内原子提交。当客户端因超时发起重试时，强制复用相同的操作标识。服务端收到重试请求后，优先检索幂等记录：若命中已提交记录，则直接返回原事务执行结果，避免重复执行掷骰或数据变更；若检测到相同标识正在被并发事务处理，则提示客户端稍后重试。该机制以服务端原子事务与幂等记录为支撑，保证了网络超时重试时的操作安全性。其生效前提在于：客户端必须生成并维护全局唯一的操作标识，服务端在原子事务中保证操作与结果记录的持久性。

(2) 客户端断线重连与增量/全量快照差分恢复流程

针对客户端断线重连，系统设计了差分状态恢复流程。图 6 展示了客户端断线重连与授权状态恢复的逻辑流转。

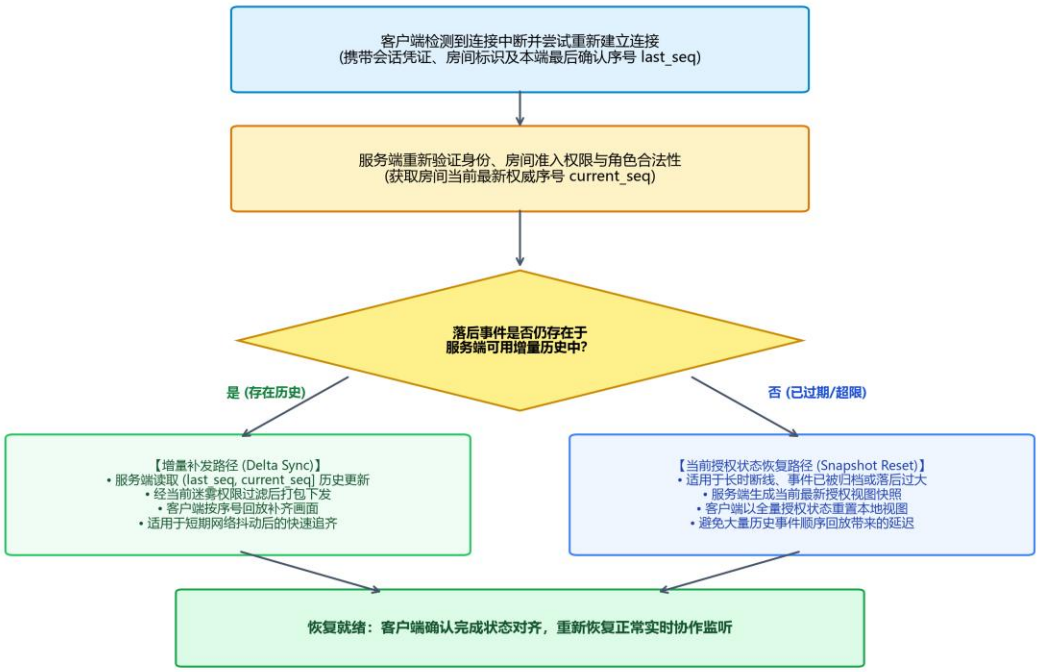


图 6　客户端断线重连与授权状态恢复流程图

结合图 6，系统对重连恢复的分支判定推导如下：（1）重连发起与准入重新核验：客户端检测到底层网络连接断开后，启动指数退避重连。重新建连后发送重连信令，携带身份凭证、房间标识以及本端最后成功处理的事件序号；服务端首先重新验证该用户的房间准入与权限（防止断网期间被移出房间）；（2）差分恢复决策分流：服务端对比房间最新权威事件序号与客户端上报序号：① 增量补发路径（Delta Sync，轻度落后）：若落后步数处于服务端保留的历史事件窗口内，服务端直接检索落后区间的有序增量事件列表，经当前战争迷雾视野权限过滤后，打包推回给客户端按序回放补齐画面，网络传输开销较低；② 全量快照重置路径（Snapshot Reset，长时间离线）：若客户端断线时间过长超出历史保留窗口，服务端则调用状态序列化组件生成当前时刻的房间授权状态基准快照（Baseline Snapshot）。需要特别澄清的是，本系统所采用的快照恢复机制，本质上是由集中权威服务导出的房间状态截面与历史重放，并不等同于在对等无中心网络中运行的经典 Chandy–Lamport 分布式全局状态快照算法 [11]（后者需通过通道标记 Marker 机制协调无集中协调者的分布式全局状态截面）；引入文献 [11] 仅用于建立与分布式系统课程中全局状态截面概念的理论对照与学术讨论，系统并未实现亦不需要去中心化的 Chandy–Lamport 算法。快照包含预置地图标识、当前所有可见标记及其属性，客户端接收后基于基准快照重置本地图形上下文；（3）就绪确认：无论采用增量补发还是全量快照重置，客户端完成状态同步后均向服务端发送同步就绪确认帧，重新切入正常协同状态。系统将‘恢复时间 ≤5 秒’作为设计目标用于指导差分恢复策略设计，该指标受网络环境与快照大小制约，属于未经测试的设计假设与目标，并非实测性能。

表 8-1 典型网络与节点故障场景及恢复策略分析表

典型故障场景	根本故障成因与物理表现	系统预期容错与自愈策略	涉及课程理论与知识点
客户端网络突发闪断	无线网络信号抖动，长连接异常断开	客户端指数退避重连；服务端保留短期会话并走增量补发或快照重置	Chapter 4 瞬态通信缺陷；Chapter 8 崩溃恢复模型
请求提交成功但响应丢失	上行成功入库，下行网络拥塞引发客户端超时	客户端复用 operationId 重试，服务端命中幂等记录直接返回原结果	Chapter 1 事务边界；Chapter 8 RPC 幂等重试语义
房间协作服务实例异常退出	宿主机故障或协作进程崩溃	路由网关检测心跳超时，备用实例从持久化存储拉取快照接管服务	Chapter 3 进程故障；Chapter 8 进程韧性与故障切换
实时增量事件网络丢包	长连接链路偶发丢帧或网关队列短暂溢出	客户端感知到 sequence 不连续，主动发起增量补发拉取请求	Chapter 4 成组通信；Chapter 7 顺序一致性与单调性
持久化存储短暂不可用	主数据库触发主备切换或瞬时磁盘 I/O 阻塞	协作层暂停该房间的新写事务，向前端返回降级重试提示	Chapter 1 ACID 持久性边界；Chapter 7 复制一致性

(3) 协作服务进程故障接管与恢复前提边界

当负责某房间的协作服务实例发生故障时，系统依靠共享持久化存储进行状态恢复与接管。在此需要阐明故障接管的安全前提与潜在风险：必须防范‘假死脑裂（Split-Brain）’现象。假设服务实例 1 因网络抖动或长垃圾回收停顿与网关失去联系，网关判定其离线并将该房间的协调职责分配给备用实例 2。若实例 1 随后恢复运行并未感知到已被替代，若继续向持久化存储写入数据，将造成多主并发写冲突与数据混乱。因此，故障接管机制在理论上必须引入带有租约（Lease）与递增任期号（Epoch / Fencing Token）的存储锁与准入隔离机制 [1, 2]：持久化存储在执行写入时强制核验任期标识，新实例接管时递增任期号，旧实例携带过期任期号发起的任何写操作将被底层存储拒绝，以此降低脑裂风险。需要特别指出的是，本报告对服务实例接管与隔离机制的讨论属于概要设计层面的理论推导与安全模型约束，在工程实践中依赖严格的时钟漂移控制、心跳超时阈值设定与存储层强原子条件更新能力，属于设计目标，尚未在系统中实现完整的自动化接管测试。

(4) 课件理论分析：故障分类模型、可靠通信与 2PC/3PC 适用边界

对照宋杰老师课件 Chapter 8（Fault Tolerance）关于容错理论与分布式提交协议的权威论述 [1]，系统建立如下理论映射：

1. 故障模型界定（Fault Models）：课件将分布式系统故障划分为崩溃故障（Crash-Stop）、崩溃恢复故障（Crash-Recovery）、省略故障（Omission Failure）、定时故障（Timing Failure）以及拜占庭故障（Byzantine Failure / Arbitrary Failure）。雾桌平台将系统容错边界明确界定在‘非拜占庭的崩溃-恢复故障与网络省略故障’范围内：假定服务端进程与数据库发生故障后会重启恢复，网络消息可能丢包或延迟，但服务端节点内部假定不存在伪造恶意数据的拜占庭叛徒行为。对于客户端可能存在的伪造请求或作弊嗅探，系统通过严格的服务端鉴权与战争迷雾差分投影加以防御，而非在系统内部引入开销巨大的拜占庭容错协议。

2. 可靠通信与 RPC 失败语义（RPC Semantics Under Failure）：课件分析了 RPC 面临

节点崩溃或消息丢失时的常见语义：至少一次（At-least-once）、至多一次（At-most-once）与恰好一次（Exactly-once）。在不可靠的物理网络环境下，网络消息本身无法天然保证只投递一次。雾桌平台在网络层采用带有超时的重试机制（体现‘至少一次’通信倾向），在应用层结合唯一 `operationId` 幂等记录与事务更新，从而在端到端应用语义层面逼近‘恰好一次（Effectively-once / Idempotent Execution）’的处理效果，其本质是‘至少一次传输 + 幂等去重’，而非物理网络保证消息只投递一次。

3. 分布式两阶段提交（2PC）与三阶段提交（3PC）适用性论证：课件 Chapter 8 推导了两阶段提交（Two-Phase Commit, 2PC）与三阶段提交（3PC）在保障跨节点分布式事务原子性方面的机制与限制 [1, 10]。

设计论证：2PC/3PC 适用于‘单个逻辑业务操作需要跨越多个独立的分布式资源管理器（如多个独立数据库或消息队列）保持原子提交’的场景。其代价是协调者单点阻塞、长时持有锁资源以及网络分区下的阻塞风险。在雾桌平台的概要设计中，我们将同一房间内的状态变更、增量事件生成与操作幂等记录，归集在‘同一个权威持久化数据库实例的本地事务’中完成。由于不涉及跨独立数据库的分布式事务参与者，底层数据库自身的预写式日志（WAL）与事务隔离机制足以保障单个房间写操作的 ACID 特性。因此，在现阶段的单数据库权威存储架构下，引入跨节点 2PC/3PC 缺乏业务必要性，属于过度设计；本设计明确界定其适用边界，仅将其作为未来数据分片或跨服务微服务拆分时的可选演进方案。

9. 总结

9.1 系统设计与分布式知识点对应回顾

本报告针对选题 B‘多人实时协作的在线虚拟桌游平台’，完成了一套面向课程要求的分布式系统概要设计说明书。系统紧扣分布式系统的核心特征与工程权衡展开设计，涵盖了《分布式系统导论》课程教学大纲与宋杰老师课件中 Chapter 1 至 Chapter 8 的核心理论模块 [1]，尝试将理论概念与业务架构建立对应。课程核心知识点在系统设计中的应用与映射如表 9-1 所示。

表 9-1 课程 Chapter 1–8 知识点在系统设计中的应用与映射对应表

课程课件章节	核心概念与理论依据	本系统概要设计中的具体映射与落地	不采用或边界限定说明
Chapter 1 Introduction	分布式目标、透明性、陷阱、ACID 事务	定义网络延迟假定；以单库本地事务为边界；透明性与迷雾隔离权衡	不把数据库提交成功等同于网络广播送达
Chapter 2 Architectures	分层架构、C/S、P2P 对等网络、事件驱动	分层客户端—服务器架构；按房间分配权威协作；事件驱动异步通知	分析并排除全 P2P 架构（基于防作弊与迷雾要求）
Chapter 3 Processes	线程模型、并发处理、客户端分工、代码迁移	瘦客户端交互、服务端权威排队；异步事件驱动思路	论证桌游业务无需动态代码迁移，予以排除
Chapter 4 Communication	RPC、瞬态通信、持久通信、面向流通信	REST 接口与全双工长连接；增量事件持久化存储记录	明确判定长连接/广播通知为瞬态，恢复依赖持久化记录

课程课件章节	核心概念与理论依据	本系统概要设计中的具体映射与落地	不采用或边界限定说明
Chapter 5 Naming	平面命名、结构化命名、邀请凭证、DHT/Chord	UUIDv4 实体全局唯一（非安全授权）；邀请短码哈希；路由目录	强调随机 ID 不能替代权限验证；百级规模排除自建 Chord
Chapter 6 Synchronization	物理时钟偏差、Lamport 时钟与权威定序区分、互斥算法	单房间权威单调事件序号定序 + 乐观版本校验；Lamport 时钟阐释因果序	明确区分理论因果序与服务端权威序号；说明 Ricart-Agrawala 2(N-1) 消息开销过大并排除
Chapter 7 Consistency	顺序一致性、读自身写与乐观预览区分、主备复制与 Quorum	核心数据顺序一致；已提交写保障读自身写；前端推测预览解耦；迷雾零信任投影	采用成熟主备异步复制候选；指明等权 Quorum (R+W>N, W>N/2) 与加权模型的区别及适用边界
Chapter 8 Fault Tolerance	故障模型、RPC 幂等、崩溃恢复、2PC/3PC	唯一操作标识幂等记录；差分状态恢复流程；租约防脑裂讨论	单库事务自足，论证当前阶段为何无需引入 2PC/3PC

9.2 核心设计取舍与工程理性总结

纵观全篇报告，本设计围绕实际业务需求与分布式系统基本原理展开推演，体现了以下设计取舍与思考：（1）在体系结构选择上：基于多人在线桌游存在严格信息不对称（战争迷雾）与防作弊要求的业务特点，报告论证了全 P2P 架构在客户端作弊防范与弱网一致性上的局限，确立了分层客户端—服务器架构与单房间权威协调机制；（2）在并发控制与同步上：避免在客户端复杂堆叠分布式互斥锁或多主 CRDT 算法 [13]，采用单房间权威排队与乐观版本号校验，实现了较为简洁的冲突检测与安全回滚流程，降低了系统整体协调开销；（3）在通信与容错设计上：区分了瞬态通信通道与持久化状态记录的职责差异，认识到‘网络消息发送成功不等于客户端接收’以及‘网络请求超时不等于操作执行失败’的客观规律，结合唯一 `operationId` 幂等机制与差分状态恢复策略构建容错逻辑；（4）在安全边界防范上：确立零信任客户端原则，通过服务端差分投影机制限制未授权数据下发，有效规避了客户端内存窃取战争迷雾底层数据的作弊风险。

9.3 设计局限性与未验证假设声明

遵循严谨求实的科学态度，报告对本设计的局限性与待验证项作出如下客观说明：（1）性能指标属于未经测试的设计假设与目标：单房间最多 8 人、初始约 100 个并发房间、P95 延迟目标 ≤300ms（权威库提交至授权客户端）与常规 API 处理 P95 延迟 ≤500ms，以及网络恢复后普通房间 5 秒内恢复状态等指标，均为用于辅助系统推演的设计假设或设计目标，不是教师规定的硬性指标，也不是已经实现或经过压力测试验证的性能承诺，实际表现受限于网络环境、硬件配置与并发争用程度；（2）异步复制下的数据丢失风险：若持久化存储采用主备异步流复制机制，在主节点发生突发硬件掉电或严重故障时，尚未复制到备用节点的数据存在丢失风险，系统不承诺绝对的‘零数据丢失’；（3）服务接管机制尚未经过完整工程实现验证：虽然提出了租约与 Fencing Token 的防脑裂思路，但自动故障切换在实

际生产中对心跳阈值、时钟漂移和网络分区的容忍度要求极高，本报告仅给出了概要设计方案，尚未完成详细工程实现与极端测试；（4）单房间权威排队与系统水平扩展的权衡：单房间由单一权威节点协调简化了并发控制，但若未来出现超大规模房间或极高频标记更新，该单点可能成为局部吞吐瓶颈，需在后续详细设计中进一步评估水平切分策略。上述局限性为系统后续的工程原型实现、性能基准测试与详细架构设计提供了明确的方向。

参考文献

[1] 宋杰. 分布式系统导论 (Introduction to Distributed System) [Z]. 东北大学软件学院教学课件, 2024.

[2] TANENBAUM A S, VAN STEEN M. Distributed systems: principles and paradigms [M]. 2nd ed. Upper Saddle River: Pearson Prentice Hall, 2007.

[3] LAMPORT L. Time, clocks, and the ordering of events in a distributed system [J]. Communications of the ACM, 1978, 21(7): 558-565.

[4] GIFFORD D K. Weighted voting for replicated data [C]//Proceedings of the seventh ACM symposium on Operating systems principles. 1979: 150-162.

[5] RICART G, AGRAWALA A K. An optimal algorithm for mutual exclusion in computer networks [J]. Communications of the ACM, 1981, 24(1): 8-17.

[6] MATTERN F. Virtual time and global states of distributed systems [C]//Parallel and Distributed Algorithms. North-Holland, 1989: 215-226.

[7] FIELDING R T. Architectural styles and the design of network-based software architectures [D]. Irvine: University of California, Irvine, 2000.

[8] FETTE I, MELNIKOV A. The WebSocket Protocol [R]. RFC 6455, 2011.

[9] Owlbear Rodeo. Owlbear Rodeo: A virtual tabletop for tabletop role-playing games [EB/OL]. (2020) [2026-10-23]. https://www.owlbear.rodeo.

[10] GRAY J. Notes on data base operating systems [M]//Operating Systems. Berlin: Springer, 1978: 393-481.

[11] CHANDY K M, LAMPORT L. Distributed snapshots: determining global states of distributed systems [J]. ACM Transactions on Computer Systems, 1985, 3(1): 63-75.

[12] VOGELS W. Eventually consistent [J]. Communications of the ACM, 2009, 52(1): 40-44.

[13] SHAPIRO M, PREGUIÇA N, BAQUERO C, et al. Conflict-free replicated data types [C]//Symposium on Self-Stabilizing Systems. Berlin: Springer, 2011: 386-400.

[14] STOICA I, MORRIS R, KARGER D, et al. Chord: A scalable peer-to-peer lookup service for internet applications [J]. ACM SIGCOMM Computer Communication Review, 2001, 31(4): 149-160.

[15] BIRMAN K P, JOSEPH T A. Reliable communication in the presence of failures [J]. ACM Transactions on Computer Systems, 1987, 5(1): 47-76.

附录 (得分栏勿填)

组名	系统名	组长			副组长 1			副组长 2			副组长 3			组员 1			组员 2			组员 3			组员 4		
		姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分
完成作业组	雾桌	周宇斌	20246085		刘润七	20246162		刘万轩	20246063		周星宇	20245986													